

CS253 Assignment 1: Memory-Efficient Versioned File Indexer Using C++

Aditi

Roll Number: 240048

1 Introduction / Overview

The assignment requires designing and implementing a **memory-efficient versioned file indexer** in C++. The program processes large text files incrementally using a fixed-size buffer and creates a **word-level index** for analytical queries. The primary objectives are:

- Avoid loading the entire file into memory.
- Support multiple versions of a file.
- Allow queries such as word count, top-K frequent words, and differences between versions.
- Demonstrate object-oriented programming, inheritance, polymorphism, templates, and exception handling.

2 Design and Approach

The program follows a modular, object-oriented design with clearly defined responsibilities for each class. The overall approach includes:

1. **Buffered File Reading:** Read files incrementally using a fixed-size buffer (256 KB to 1024 KB) to avoid high memory usage.
2. **Tokenization:** Convert input chunks to lowercase and extract words while handling cases where a word is split across buffers.
3. **Versioned Indexing:** Maintain a mapping from words to their frequencies for each file version.
4. **Query Execution:** Support three query types:
 - Word Count Query
 - Difference Query
 - Top-K Query
5. **Command-Line Interface:** Allow users to specify input files, versions, buffer size, query type, and additional parameters.

3 Classes Used

3.1 BufferedFileReader

Handles reading files in chunks of a fixed buffer size. Ensures that no more than the buffer size is loaded into memory at a time. **Key functions:**

- `BufferedFileReader(path, size)`: Constructor opens the file and allocates buffer.
- `readChunk(outData)`: Reads the next chunk into a string.

3.2 Tokenizer

Splits chunks of text into words and handles the case where a word is split between two chunks. Converts all characters to lowercase for case-insensitive processing. **Key functions:**

- `tokenize(chunk)`: Returns a vector of words from the chunk.
- `flushLeftover()`: Returns the last incomplete word, if any.

3.3 VersionedIndexer

Stores the word-level index for multiple file versions. Supports adding words, querying word counts, computing differences, and retrieving top-K frequent words. **Key functions:**

- `addWord(version, word)`: Increment word count for a version.
- `getWordCount(version, word)`: Returns frequency of a word in a version.
- `diffWord(v1, v2, word)`: Computes difference between two versions.
- `topK(version, k)`: Returns top-K frequent words.

3.4 Query and Derived Classes

Implements runtime polymorphism for different query types:

- `Query`: Abstract base class with pure virtual function `execute()`.
- `WordQuery`: Executes word count queries.
- `DiffQuery`: Executes difference queries.
- `TopKQuery`: Executes top-K queries.

4 File Processing with Fixed-Size Buffer

The program processes files incrementally:

1. Open the file with `BufferedReader` using the specified buffer size.
2. Read the next chunk of the file into memory.
3. Pass the chunk to `Tokenizer` to extract words.
4. Add each word to the `VersionedIndexer`.
5. Handle any leftover partial word at the end of a chunk using `flushLeftover()`.

This approach ensures memory usage depends only on the number of unique words, not file size.

5 Assumptions and Observations

- Words are defined as contiguous alphanumeric sequences (A-Z, a-z, 0-9).
- Word matching is case-insensitive.
- Buffer size must be between 256 KB and 1024 KB.
- The system can handle multiple file versions in a single execution.
- Execution time depends on file size and buffer configuration; larger buffer sizes do not necessarily reduce processing time due to tokenization overhead.
- Invalid inputs (like too small or too large buffer) are handled via exceptions.